

Lesson 9 Teacher's Key: On the Scoreboard

Unity - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

The polling pattern (checking data every single frame in Update(), regardless of whether it changed) as a legitimate, simple alternative to Godot/Unreal's push-based approach - and when each is the more natural fit.

Level 1 - Full Worked Solution

- HUD.cs is a small, separate script whose only job is displaying data - it never changes score or lives, only reads them from GameManager every frame.
- Because Update() runs every frame unconditionally, there's no need to remember to call a refresh function anywhere else in the codebase - the tradeoff is doing slightly more work than strictly necessary (checking even when nothing changed), which is harmless for two small ints.
- GameManager.cs continues to be the single source of truth for score/lives - HUD.cs only reads it, never writes it.

Highlighted lines are new this week; everything else is code you already have.

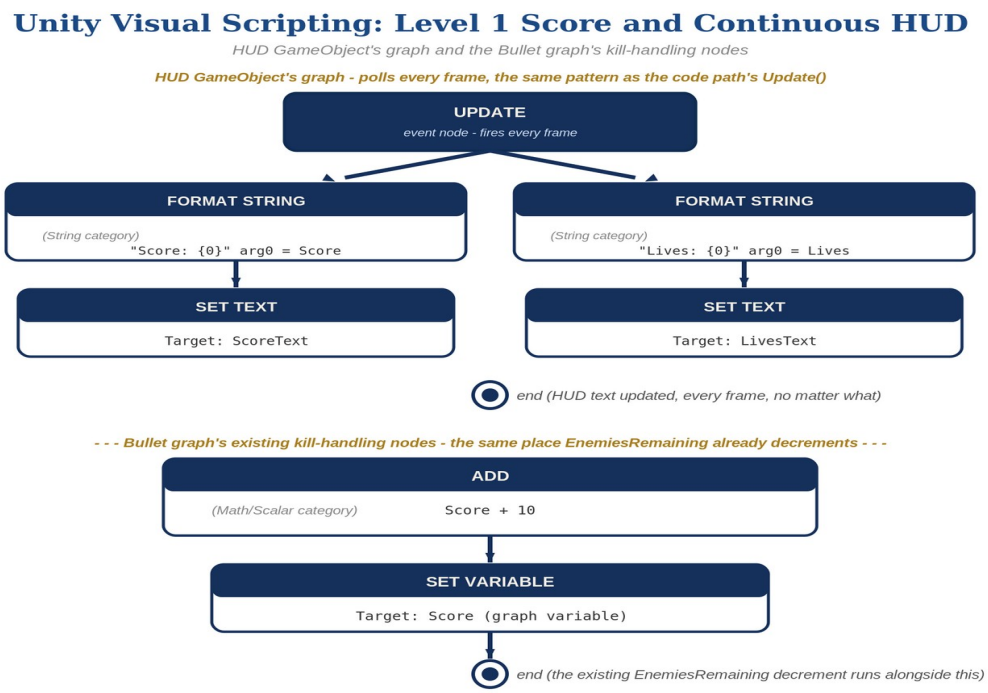
```
// HUD.cs (new script, attached to the HUD GameObject):
using UnityEngine;
using UnityEngine.UI;

public class HUD : MonoBehaviour {
    public Text scoreText;
    public Text livesText;

    void Update() {
        scoreText.text = "Score: " + GameManager.score;
        livesText.text = "Lives: " + GameManager.lives;
    }
}

// GameManager.cs - the new field and the revised RegisterKill():
public static int score = 0;

public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    if (enemiesRemaining <= 0) hasWon = true;
}
```



The HUD GameObject's graph and the Bullet graph's kill-handling nodes - the Visual Scripting equivalent of this solution, for students comparing both paths.

Why This Structure

- Polling is simpler to reason about for beginners specifically because there's no need to identify every single place data changes and remember to call a refresh - a real advantage for a script that only needs to display two cheap values.
- This contrasts usefully with GameManager's own RegisterKill()/RegisterEscape() methods, which ARE event-driven (called only when something happens) - the lesson isn't "polling is better," it's "different jobs suit different approaches."

Level 2 / Level 3 Talking Points

- Ask students: would polling still make sense if the HUD needed to update 500 different values every frame? (Probably not - this is a good entry point for why push-based approaches exist and matter at larger scale, previewing ideas they'll meet again in more advanced programming.)
- For students who did Godot too: have them explicitly compare update_hud() (called only when needed) against HUD.cs's Update() (called unconditionally, every frame) - both are correct, valid solutions to the same problem.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: 3 kills read exactly "Score: 30"; an escape moves lives but
```

```
never score;
// both numbers are correct the instant the game starts, not only after
the first kill or escape.
```

Level 3 - complete code (all goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// GameManager.cs - Level 3 RegisterKill():
public static void RegisterKill() {
    enemiesRemaining--;
    score += Random.Range(5, 16);
    if (enemiesRemaining <= 0) hasWon = true;
}

// HUD.cs - Level 3 visual (or set these in the Inspector instead):
void Start() {
    scoreText.fontSize = 28;
    scoreText.color = Color.yellow;
}
```

Common Student Mistakes

- Forgetting to assign ScoreText/LivesText in the Inspector after writing HUD.cs - causes a NullReferenceException the instant Play starts.
- Declaring score as a non-static field on GameManager by mistake, breaking the static-class pattern established since Lesson 6.
- Placing the new UI Text objects outside the Canvas (rare, but possible if a Canvas wasn't auto-created correctly) - text won't render if this happens.